

DOKUMEN TEKNIS RESMI • V1.0.0

UNIVERSITAS ACHMAD YANI (UAY)

UAY FILE SERVICE

DOKUMENTASI RESMI REST API

Panduan Komprehensif Arsitektur Otorisasi OIDC/Keycloak, Matriks 12 Hak Akses Granular, Pipeline Validasi Biner, Streaming Media RFC 7233, dan Katalog Endpoint Lengkap.

KLASIFIKASI DOKUMEN

Technical & API Integration Spec

TANGGAL EFEKTIF

2 Oktober 2026

KERNEL BACKEND

NestJS / Node.js • Fastify Engine

OTORITAS PENERBIT

Biro Sistem Informasi & TI Kampus UAY

Daftar Isi Dokumen

1. Arsitektur & Konvensi Global	Hal. 2	2. Mekanisme Autentikasi & Identitas	Hal. 3
3. Matriks Hak Akses (12 Permissions)	Hal. 4	4. Standar Format Respons Global	Hal. 5
5. Kebijakan Validasi Biner & Batasan	Hal. 5	6. Katalog Endpoint Lengkap	Hal. 6
7. Protokol Streaming Media RFC 7233	Hal. 10	8. Siklus Hidup Berkas & Auto-Purge	Hal. 11
9. Kode Status HTTP & Error Handling	Hal. 12	10. Contoh Integrasi Kode Aplikasi Klien	Hal. 12

1. Arsitektur & Konvensi Global

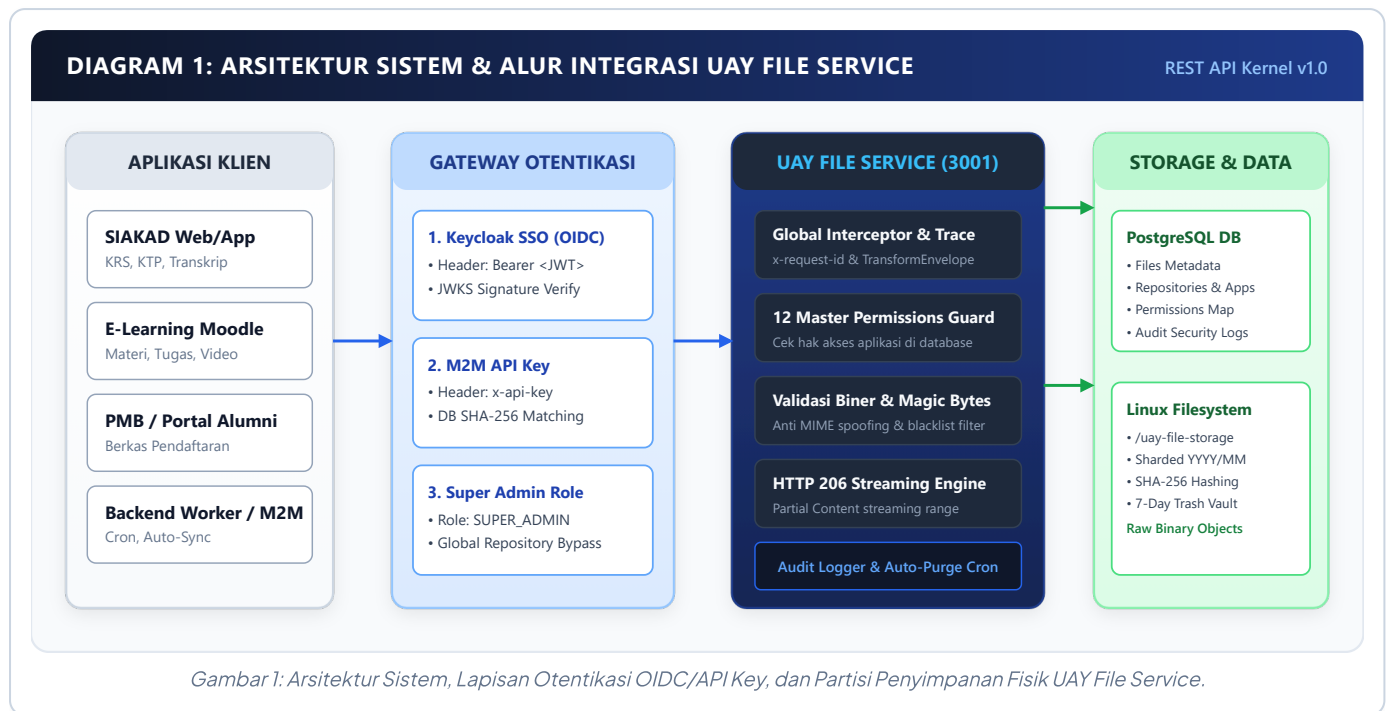
UAY File Service merupakan layanan microservice penyimpanan berkas terpusat yang dirancang untuk memenuhi kebutuhan seluruh ekosistem aplikasi Universitas Achmad Yani (SIKAD, E-Learning, PMB, Tracer Study Alumni, dan Background Worker Kampus). Layanan ini menyediakan API bervolume tinggi, terotentikasi, dan aman dengan integritas data berbasis SHA-256.

Base URL Layanan

Lingkungan	URL Basis (Base URL)	Tujuan Penggunaan
Lokal (Development)	http://localhost:3001	Pengujian dan perakitan fitur lokal
Jaringan Kampus / Intranet	http://file-service.uay.ac.id	Staging dan produksi kampus UAY
Interactive Swagger UI	http://localhost:3001/api/docs	Eksplorasi OpenAPI & interaktif testing

Header Standar Komunikasi API

Header Name	Tipe Data	Status	Keterangan & Fungsi
Authorization	String	WAJIB (USER)	Bearer JWT Token yang diterbitkan oleh Keycloak SSO UAY.
x-api-key	String	ALTERNATIF	API Key untuk komunikasi <i>Machine-to-Machine</i> (M2M) backend/cron.
x-client-id	String	OPSIONAL	Identifer Client ID saat pengujian lokal atau emulasi klien.
x-actor-id	String	OPSIONAL	User ID / Username pengguna akhir yang memicu aksi untuk audit trail.
x-request-id	UUID	OPSIONAL	Unique Trace ID untuk pelacakan transaksi antar-microservice.



2. Mekanisme Autentikasi & Identitas Klien

Kernel otorisasi UAY File Service mengidentifikasi dan memvalidasi pemanggil API melalui tiga mekanisme terisolasi:

Metode 1: OIDC Bearer Token (Rekomendasi untuk Web / Mobile Client)

Digunakan oleh aplikasi frontend yang berinteraksi langsung dengan civitas akademika:

```
Authorization: Bearer <access_token_jwt>
```

File Service secara otomatis memvalidasi tanda tangan kriptografi token melalui endpoint JWKS Keycloak kampus, serta mengekstrak klaim identitas `client_id` (atau `azp`) dan aktor penanggungjawab (`preferred_username` / `email`).

Metode 2: Machine-to-Machine (M2M) API Key

Didesain khusus untuk microservice backend internal, cron jobs, atau ETL data tanpa sesi pengguna aktif:

```
x-api-key: uay_sec_live_9f82a170bc484e1b...
```

API Key dicocokkan dengan hash SHA-256 yang tersimpan di tabel basis data aplikasi. Sistem akan menetapkan identitas aplikasi serta hak akses permission yang telah didelegasikan secara spesifik.

Metode 3: Hak Akses Super Administrator

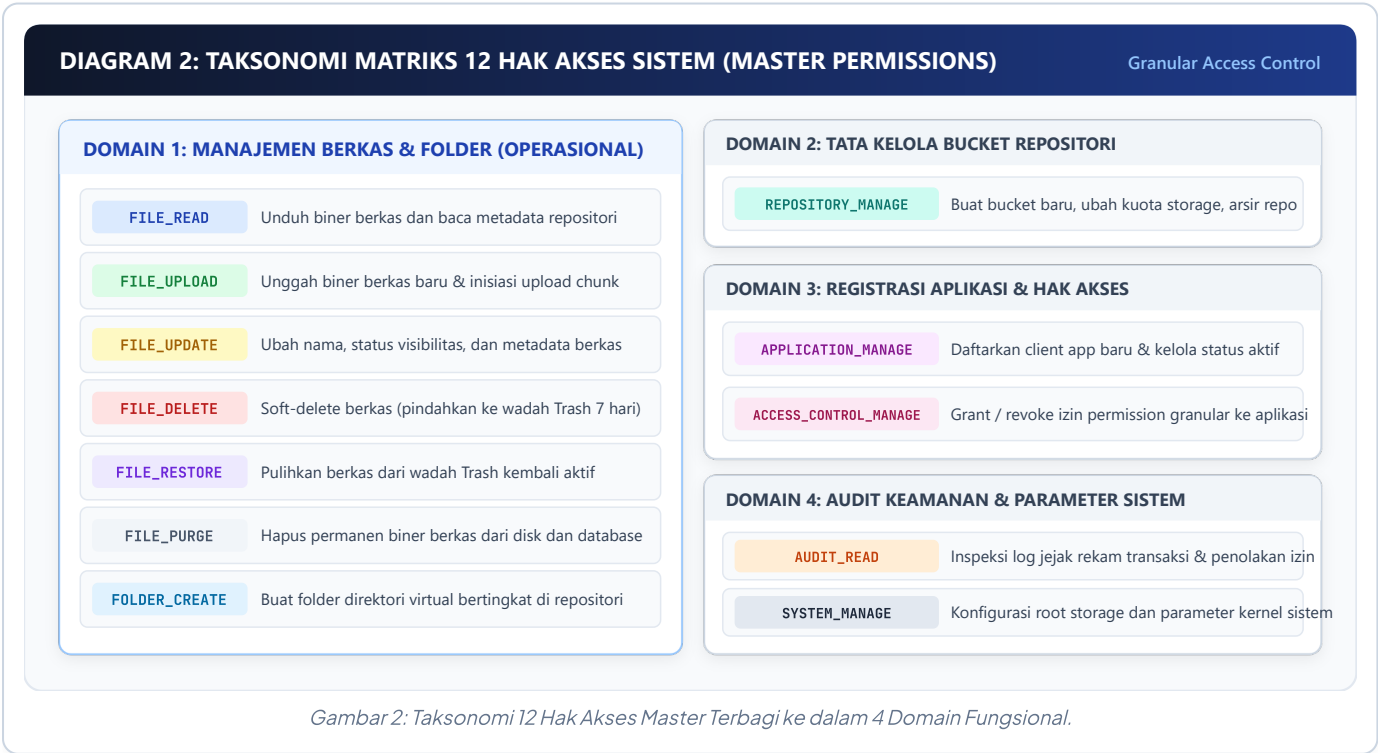
Jika token otentikasi memiliki peran `SUPER_ADMIN`, username `superadmin`, atau menggunakan secret key `uay-superadmin-secret`:

- **Global Bypass:** Dapat membaca, mengubah, dan mengelola seluruh bucket repositori milik semua aplikasi tanpa hambatan kepemilikan.
- **Master Authority:** Memiliki seluruh 12 izin sistem secara otomatis untuk pemeliharaan dan audit darurat.

3. Matriks Hak Akses (12 Master Permissions)

Setiap aplikasi klien yang terdaftar di basis data memiliki kumpulan izin granular (*permission*) yang dapat diatur melalui antarmuka **Access Control**:

Permission Code	Nama Izin Resmi	Keterangan Otoritas & Batasan
FILE_READ	Read and Download	Mengunduh biner berkas dan membaca metadata repositori yang diizinkan.
FILE_UPLOAD	Upload Files	Mengunggah biner berkas baru dan membuat subfolder direktori.
FILE_UPDATE	Update Files	Mengubah nama berkas, status visibilitas, dan merestorasi berkas dari Trash.
FILE_DELETE	Delete Files	Melakukan <i>soft-delete</i> berkas (memindahkan ke wadah Trash retensi 7 hari).
FILE_RESTORE	Restore Files	Memulihkan berkas dari wadah Trash kembali ke status aktif.
FILE_PURGE	Purge Files	Menghapus permanen biner berkas dari disk fisik Linux dan basis data PostgreSQL.
FOLDER_CREATE	Create Folders	Membuat direktori virtual bertingkat di dalam repositori aplikasi.
REPOSITORY_MANAGE	Manage Repositories	Membuat bucket baru, mengubah kuota penyimpanan, dan mengarsip repositori.
APPLICATION_MANAGE	Manage Applications	Mendaftarkan aplikasi klien baru dan mengubah status operasional aplikasi.
ACCESS_CONTROL_MANAGE	Manage Access Control	Mengatur penugasan dan pencabutan izin granular ke aplikasi klien.
AUDIT_READ	Read Audit Logs	Mengakses catatan jejak rekam audit trail keamanan dan log operasional.
SYSTEM_MANAGE	Manage System	Mengubah konfigurasi path root storage dan parameter kernel server.



4. Standar Format Respons (Global Envelope)

Seluruh respons JSON dari UAY File Service dibungkus secara konsisten menggunakan NestJS `TransformInterceptor` :

Contoh Respons Sukses (HTTP 200/201):

```
{
  "statusCode": 200,
  "success": true,
  "data": {
    "fileId": "65b90f42-491a-47f3...",
    "originalName": "laporan.pdf",
    "sizeBytes": 2457810,
    "status": "active"
  },
  "message": "Operasi berhasil",
  "timestamp": "2026-10-02T11:25:00Z"
}
```

Contoh Respons Galat (HTTP 403 Forbidden):

```
{
  "statusCode": 403,
  "success": false,
  "error": "ForbiddenException",
  "message": "Aplikasi tidak memiliki izin: [FILE_UPLOAD]",
  "timestamp": "2026-10-02T11:25:00Z",
  "path": "/api/v1/files"
}
```

5. Kebijakan Validasi Biner & Batasan Ukuran

Untuk menjaga keamanan server kampus dari serangan *malware* dan *spoofing*, diterapkan 5 tahap validasi:

- 1. **Blacklist Ekstensi Berbahaya:** Berkas eksekusi (`.exe` , `.sh` , `.bat` , `.cmd` , `.dll` , `.vbs` , `.js` , `.apk` , `.php` , `.py` , `.html` , dll.) langsung ditolak seketika (HTTP 400).
- 2. **Inspeksi Header Biner (Magic Numbers):** Pustaka internal membaca byte awal berkas biner secara langsung guna mendeteksi penyamaran ekstensi (*MIME spoofing*), misal executable yang di-rename menjadi `.pdf` (HTTP 415).
- 3. **Batas Ukuran Maksimal Kategori Berkas:**

Kategori	Ekstensi Format Diakui	Batas Maksimum
IMAGE	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.webp</code>	5 MB
PDF	<code>.pdf</code>	20 MB
DOC / DOCX	<code>.doc</code> , <code>.docx</code>	15 MB
XLS / XLSX	<code>.xls</code> , <code>.xlsx</code> , <code>.csv</code>	15 MB
PPT / PPTX	<code>.ppt</code> , <code>.pptx</code>	30 MB
ZIP / ARSIP	<code>.zip</code> , <code>.rar</code> , <code>.tar.gz</code>	30 MB
AUDIO	<code>.mp3</code> , <code>.wav</code> , <code>.ogg</code>	20 MB
VIDEO	<code>.mp4</code> , <code>.mkv</code> , <code>.webm</code>	50 MB
OTHER	Dokumen teks bebas non-eksekutabel	10 MB

- 4. **Integritas Data SHA-256:** Setiap berkas yang tersimpan dihitung *hash checksum* otomatis untuk deteksi korupsi biner.



GET /api/v1/files/:id

Mengunduh biner berkas atau memutar streaming media di peramban (mendukung byte range HTTP 206).

Otorisasi: Public jika `visibility = public`; Memerlukan Token jika `private`

Headers: Mendukung `Range: bytes=start-end`

```
# Contoh Download Langsung:
curl -O -J http://localhost:3001/api/v1/files/65b90f42-491a-47f3-ba78-98e9b62649b1 \
-H "Authorization: Bearer <TOKEN>"
```

GET /api/v1/files/:id/metadata

Membaca detail berkas (ukuran, checksum SHA-256, visibilitas, status) tanpa mengunduh biner fisik. Wajib izin: `FILE_READ`.

PATCH /api/v1/files/:id

Mengubah nama berkas (*rename*), status visibilitas publik/privat, atau memindahkan berkas ke folder lain.

Izin Wajib: `FILE_UPDATE` **Content-Type:** `application/json`

```
# Body JSON:
{
  "original_name": "laporan_revisi_final.pdf",
  "visibility": "public",
  "folder_id": "c138b556-9e96-48ec-8f43-41bbd0673322"
}
```

DELETE /api/v1/files/:id**POST /RESTORE**

DELETE: Memindahkan berkas ke wadah Trash (Soft-Delete) dengan masa retensi 7 hari. Wajib izin: `FILE_DELETE`.

POST /restore: Mengembalikan berkas yang berada di tempat sampah kembali ke status aktif. Wajib izin: `FILE_UPDATE`.

B. Manajemen Bucket Repositori (Repositories)

GET /api/v1/repositories

Mengambil daftar seluruh bucket repositori yang menjadi hak milik aplikasi pemanggil. Wajib izin: `FILE_READ`. Query params: `page`, `limit`.

POST /api/v1/repositories

Membuat bucket repositori baru untuk partisi penyimpanan berkas aplikasi. Wajib izin: `FILE_UPLOAD` atau `REPOSITORY_MANAGE`.

```
{
  "name": "Materi Perkuliahan 2026",
  "slug": "materi-perkuliahan-2026",
  "description": "Bucket repositori modul dan slide kuliah"
}
```

GET /api/v1/repositories/:id/files

Melihat daftar berkas aktif di dalam suatu repositori dengan opsi filter folder virtual (`?folderId=...`). Wajib izin: `FILE_READ`.

C. Direktori Folder Virtual (Folders)

POST /api/v1/repositories/:repositoryId/folders

Membuat folder virtual bertingkat di dalam repositori untuk mengelompokkan berkas secara logis. Wajib izin: `FOLDER_CREATE` atau `FILE_UPLOAD`.

```
{
  "name": "Semester Ganjil 2026",
  "parent_folder_id": null
}
```

GET /api/v1/folders/:folderId

Mengambil detail folder, daftar subfolder di dalamnya, serta total berkas yang tersimpan. Wajib izin: `FILE_READ`.

D. Tata Kelola Aplikasi Kampus (Applications)

Didedikasikan untuk Administrator guna mendaftarkan aplikasi client baru (SIAKAD, E-Learning, PMB) dan mengelola status operasionalnya.

Method	Endpoint Path	Deskripsi Fungsi
GET	/api/v1/applications	Daftar seluruh aplikasi klien yang terdaftar di basis data.
POST	/api/v1/applications	Mendaftarkan aplikasi baru (nama, slug, keycloak_client_id).
GET	/api/v1/applications/:id	Mengambil profil detail aplikasi beserta daftar izin aktif.
PATCH	/api/v1/applications/:id	Mengubah nama, deskripsi, atau mengubah status (active / inactive).
DELETE	/api/v1/applications/:id?permanent=false	Nonaktifkan aplikasi (soft) atau hapus permanen beserta izinnya.
POST	/api/v1/applications/:id/permissions	Memberikan izin granular ke aplikasi (Body: {"permission_code": "FILE_UPLOAD"}).
DELETE	/api/v1/applications/:id/permissions/:permId	Mencabut penugasan izin dari aplikasi.

E. Kamus Izin Sistem (Permissions)

GET /api/v1/permissions

Menampilkan daftar 12 izin granular yang diakui oleh kernel sistem beserta jumlah aplikasi aktif yang memilikinya.

F. Jejak Rekam Audit Keamanan (Audit Logs)

GET /api/v1/audit-logs

Kueri jejak rekam audit trail keamanan, upload berkas, hapus berkas, dan insiden penolakan otorisasi. Wajib izin:

AUDIT_READ . Filter: application_id , repository_id , action , page , limit .

G. Observabilitas & Kesehatan Sistem (Health)

GET /health | /api/v1/health

Endpoint publik tanpa token untuk probe Kubernetes / Docker Liveness & Readiness check.

GET /api/v1/health/system

Telemetri lengkap performa API, latency database PostgreSQL, sisa kapasitas storage fisik disk, dan status background job cleanup.

7. Protokol Streaming Media (HTTP 206 Partial Content)

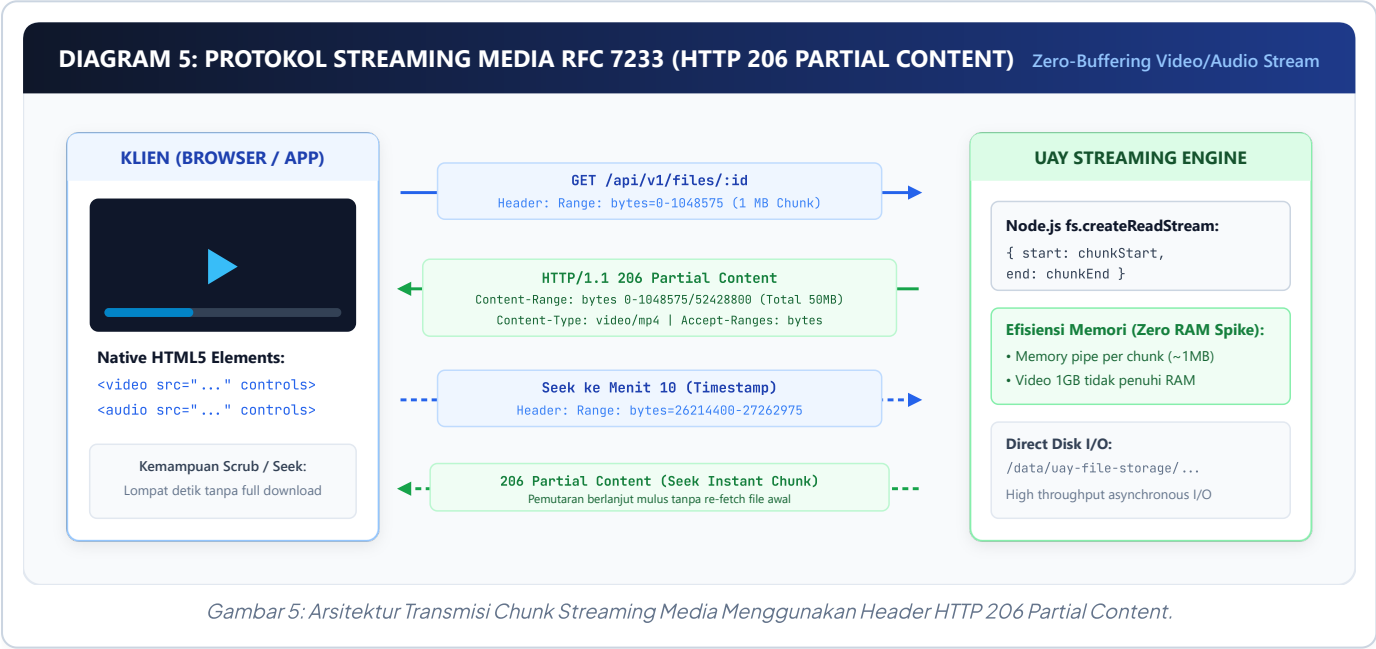
Endpoint unduh berkas `GET /api/v1/files/:id` dilengkapi dengan implementasi standar **RFC 7233** untuk pemutaran audio (`.mp3`) dan video (`.mp4`) secara efisien tanpa perlu mengunduh seluruh isi berkas ke memori RAM:

Contoh Negosiasi Header HTTP:

```
# Request dari Media Player Peramban:
GET /api/v1/files/65b90f42-491a-47f3-ba78-98e9b62649b1 HTTP/1.1
Host: file-service.uay.ac.id
Range: bytes=0-1048575

# Response dari UAY File Service:
HTTP/1.1 206 Partial Content
Content-Range: bytes 0-1048575/52428800
Content-Length: 1048576
Content-Type: video/mp4
Accept-Ranges: bytes

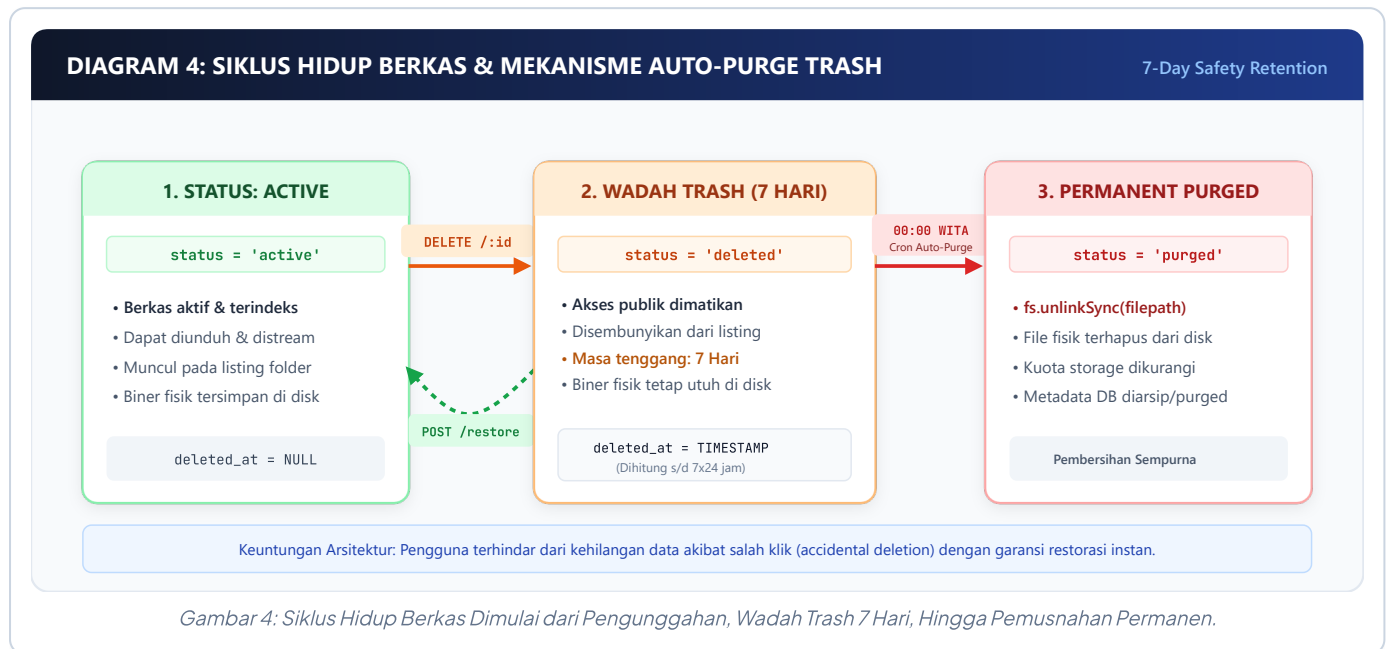
[Binary Audio/Video Stream Chunk]
```



Gambar 5: Arsitektur Transmisi Chunk Streaming Media Menggunakan Header HTTP 206 Partial Content.

8. Siklus Hidup Berkas & Auto-Purge Trash

Untuk melindungi civitas akademika dari kehilangan data yang tidak disengaja (*accidental file deletion*), UAY File Service mengimplementasikan mekanisme retensi 7 hari dengan siklus otomatis:



- Status ACTIVE:** Berkas aktif yang dapat diakses publik/privat sesuai izin dan tampil pada katalog folder repositori.
- Status SOFT DELETED (Wadah Trash):** Saat endpoint `DELETE` dipanggil, berkas ditandai dengan timestamp `deleted_at`. Berkas disembunyikan dari peramban, namun biner fisik tetap disimpan secara utuh di disk Linux.
- Proses RESTORE:** Dalam tenggang waktu 7x24 jam, pemilik berkas dapat memulihkan berkas ke posisi aslinya secara instan menggunakan endpoint `POST /restore`.
- Pembersihan Otomatis (AUTO-PURGE JOB):** Setiap hari tepat pukul **00:00 WITA**, background worker mengeksekusi pemeriksaan berkas yang telah melewati 7 hari masa tenggang, mengeksekusi `fs.unlinkSync` untuk memusnahkan biner dari hard disk, dan memperbarui status row di database menjadi `purged`.

9. Kode Status HTTP & Penanganan Kesalahan

Kode HTTP	Status Standar	Interpretasi & Panduan Solusi
200 OK	Permintaan Sukses	Permintaan pembacaan metadata, unduhan, atau pembaruan data berhasil.
201 CREATED	Entitas Baru Dibuat	Berkas fisik baru atau repositori bucket berhasil dialokasikan.
206 PARTIAL	Streaming Chunk	Pengiriman potongan biner berkas audio/video berdasarkan header <code>Range</code> .
400 BAD REQ	Parameter Tidak Valid	Form body multipart tidak lengkap atau ekstensi berkas masuk daftar blacklist.
401 UNAUTH	Belum Terotentikasi	Header token Bearer atau API Key tidak ditemukan atau kedaluwarsa.
403 FORBIDDEN	Hak Akses Ditolak	Aplikasi tidak memiliki permission yang dibutuhkan atau mengakses bucket aplikasi lain.
404 NOT FOUND	Tidak Ditemukan	ID berkas, folder, atau repositori yang diminta tidak terdaftar di sistem.
413 PAYLOAD	Ukuran Terlalu Besar	Ukuran berkas melebihi batas kuota kategori berkas yang ditetapkan.
415 MEDIA	Tipe Media Ditolak	Header biner berkas (magic bytes) tidak cocok dengan tipe MIME yang dideklarasikan.
500 SERVER	Galat Internal Server	Terjadi kendala disk I/O, konektivitas database PostgreSQL, atau service crash.

10. Contoh Integrasi Kode Aplikasi Klien

Implementasi resmi TypeScript / Node.js menggunakan `axios` dan `form-data`:

```
import axios from 'axios';
import FormData from 'form-data';
import fs from 'fs';

const API_BASE_URL = 'http://file-service.uay.ac.id/api/v1';
const CLIENT_TOKEN = 'eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...';

// 1. Fungsi Unggah Berkas Mahasiswa (Multipart Upload)
async function uploadTugasKuliah(filePath: string, repoId: string) {
  const form = new FormData();
  form.append('file', fs.createReadStream(filePath));
  form.append('repository_id', repoId);
  form.append('visibility', 'private');
  form.append('resource_type', 'tugas_kuliah');
  form.append('resource_id', 'tugas-pemrograman-01');

  const response = await axios.post(`${API_BASE_URL}/files`, form, {
    headers: {
      ...form.getHeaders(),
      Authorization: `Bearer ${CLIENT_TOKEN}`,
      'x-actor-id': 'mhs-2024-001',
    },
  });

  console.log('Upload Sukses, File ID:', response.data.data.fileId);
  return response.data.data;
}

// 2. Fungsi Pembacaan Metadata Berkas
async function getFileMetadata(fileId: string) {
  const response = await axios.get(`${API_BASE_URL}/files/${fileId}/metadata`, {
    headers: {
      Authorization: `Bearer ${CLIENT_TOKEN}`,
    },
  });

  return response.data.data;
}
```

Dukungan & Layanan Teknis

Untuk pendaftaran client ID aplikasi baru atau konsultasi integrasi microservice kampus, silakan hubungi **Biro Sistem**

Informasi & TI Universitas Achmad Yani melalui email `ti@uay.ac.id` atau kunjungi dashboard Swagger di `http://file-service.uay.ac.id/api/docs`.